

Real-Time Query Expansion & Topic Tagging System

Quick Links	
 Live Demo	https://gsdtepaqrm57qwsqzz4qu.streamlit.app/
 Frontend Repo	https://github.com/DeityAG/Real-Time-Query_Expansion_and_Topic-Tagging
 Backend Repo	https://github.com/DeityAG/Real-Time-Query_Expansion_and_Topic-Tagging_backend_pipeline/tree/main
 L1 Classifier	https://huggingface.co/Adignite/query-topic-l1-classifier
 L2 Classifier	https://huggingface.co/Adignite/query-topic-l2-classifier
 Dataset Creation	https://colab.research.google.com/drive/19faugY8dz_BTmNGw0hKpHnvibGBOGBak
 BERT Classifier	https://colab.research.google.com/drive/1P0fa2K3EOrLzw1T-E0fzUikQRAzHnJa0
 Full Inference	https://colab.research.google.com/drive/1mEkwahpPU_dU1vK_HWDXDFxEECaRtrTq

1. Problem Statement

The task is to build a system that watches a live multi-turn conversation between a user and a bot, and for every user message does two things in real time:

Two Core Tasks	
Query Expansion	Rewrite the user message into a fully self-contained question by resolving pronouns, ellipsis, and implicit references using conversation history.
Topic Classification	Assign a 2-level hierarchical topic label (e.g., Politics › India) to the (expanded) message.

The key constraint is that only the last 20 messages (10 dialogue exchanges) are available as context at inference time.

1.1 Why This Is Non-Trivial

- Pronoun references
 - "what are his duties?" → "his" refers to a named entity mentioned 2 turns ago
- Elliptical follow-ups
 - "what about uk?" → the predicate is entirely implicit from prior context
- Topic switches mid-conversation
 - Both entity and predicate change; expansion must reflect the new context
- Interruptions and small talk
 - "brb" / "wait a sec" → no expansion needed; must be tagged General
- Multi-entity comparisons
 - "compare both" → requires resolving two entities from prior turns

2. Overall Approach & Architecture

The solution consists of four stages: synthetic dataset creation, model training, inference pipeline, and a live demo. The diagram below shows the high-level pipeline at inference time.

Inference Pipeline (per user message)	
Step 1 — spaCy NER	Extracts PERSON, GPE, ORG, EVENT entities from each turn into a rolling Entity Register over the last 20 messages.
Step 2 — Interruption check	Regex + word-count heuristics detect small talk (brb, wait, ok). These are tagged General › General immediately — no LLM called.
Step 3 — LLM Rewriting	Groq API (llama-3.1-8b-instant) rewrites the user query using conversation history + entity register into a fully self-contained question.
Step 4 — DistilBERT	The expanded query is passed to two fine-tuned DistilBERT classifiers that output topic_I1 (8 classes) and topic_I2 (19 classes) with confidence scores.

2.1 Why LLM-Generated Data?

No existing dataset covers multi-turn conversational query expansion with ellipsis and pronoun resolution at this granularity. LLM generation with the 3-stage prompting framework is more practical than manual annotation at scale, and the LLM already understands context — making it the best possible annotator for expansion quality.

2.2 Why DistilBERT for Classification?

DistilBERT is 40% smaller and 60% faster than BERT-base while retaining 97% of its performance. The classification task here (~1,600 training examples, 8–19 classes) does not justify the overhead of RoBERTa or BERT-large. Two independent classifiers are trained (L1 and L2) rather than a single multi-output model, because the two label spaces have different feature requirements and separate classifiers consistently outperform shared-head approaches on short text.

2.3 Why Groq for Expansion at Inference?

Running Llama-3.2-1B locally on CPU (Streamlit Cloud free tier) takes 15–30 seconds per turn. Switching to the Groq API (llama-3.1-8b-instant) brings latency to under 1 second while using a larger, more capable model. The spaCy NER + DistilBERT components are small enough to load locally (~66 MB each) without meaningful cold-start time.

3. Dataset Creation

3.1 Challenges & Iterations

Dataset creation went through several iterations before landing on a stable approach.

1	Initial attempt — Groq llama-3.3-70b-versatile Targeted 500 samples. Rate limit hit after only 9 conversations. The 70B model combined with long system prompts consumed too many tokens per request to stay within Groq’s free-tier limits.
2	HuggingFace models on Lightning AI (L40s GPU) Tried DeepSeek-R1-Distill-Qwen-14B, Qwen3-30B-A3B, Qwen2.5-14B, Qwen3-14B. Issues: context window exceeded on 20-turn conversations, and reasoning models (R1-style) were extremely slow — 10-20 minutes per sample.
3	Groq API — llama-4-scout-17b-16e-instruct with compact prompts Reduced Stage 1 and Stage 2 prompt sizes significantly. Ran 3 separate generation scripts with checkpointing every 50 samples and 3 retries per API call. Collected ~276 valid conversations across runs.

3.2 Generation Pipeline

The 3-stage pipeline is LLM-driven throughout. NER is NOT used at dataset creation — only at inference time.

Stage	What happens	Output
Stage 1	LLM generates a 20-message alternating User/Assistant conversation seeded by (topic, entity) pair. Required to include pronouns, ellipsis, interruptions, topic switches, comparisons.	Raw conversation text
Stage 2	Separate LLM call annotates every user turn: expanded query, topic_I1, topic_I2, named entities, needs_expansion flag.	JSON array (10 objects)
Stage 3	Python checks: 20 turns, alternation, JSON valid, 10 annotations, non-empty expanded fields, topic labels in allowed set. Reject on any failure.	Valid / Rejected

3.3 Merge & Deduplication

Three separate CSV files (from the three generation runs) were merged and deduplicated using `merge_and_dedup.ipynb` at two levels:

- Conversation level — fingerprint via first-3-user-query exact match. Keeps highest-priority version on collision.
- Turn level — exact string match, then bigram BLEU ≥ 0.85 within the same topic label to catch near-paraphrases.

Raw: 344 conversations $\rightarrow$ 277 unique. Turns: 2,770 $\rightarrow$ 2,046 unique.

3.4 Final Dataset Stats

277 Conversations	2,046 User turns	79.1% Needed expansion	80 / 20 Train / Test
-----------------------------	----------------------------	----------------------------------	--------------------------------

Split	Conversations	User turns
Train (80%)	221	1,636
Test (20%)	56	410
Total	277	2,046

3.5 Topic Distribution

L1 Domain	Conversations	L2 Sub-topics
Politics	62	India, UK, USA
Sports	62	Cricket, Football, Tennis, Olympics
Technology	53	AI, Space, Gadgets
Entertainment	34	Bollywood, Hollywood
Health	28	Nutrition, Mental Health, Fitness
History	14	India, World Wars
Geography	13	Countries, Cities
General	11	Small talk / interruptions

3.6 Dataset Files & Links

- `merged_conversations.csv` — 277 rows, one per conversation (with full turns JSON, `user_queries`, `expanded_queries`, `named_entities`, `needs_expansion`)
- `turns_dataset.csv` — 2,046 rows, one per user turn, directly usable for BERT fine-tuning

4. Model Training — DistilBERT Topic Classifiers

4.1 Model Choice

Why DistilBERT over alternatives	
vs BERT-base	40% smaller, 60% faster, retains 97% of performance. Our dataset size (~1.6k) doesn't justify the overhead.
vs RoBERTa	RoBERTa's extra pre-training doesn't help on short classification queries with a fixed closed label set.
vs BERT-large	Far too large for real-time CPU inference in a Streamlit deployment.
Architecture	Two independent classifiers (L1: 8 classes, L2: 19 classes). Single multi-output head underperforms when label spaces have different feature requirements.

4.2 Training Configuration

Hyperparameter	Value
Base model	distilbert-base-uncased
Input text	expanded_query (not raw user message)
Learning rate	2e-5
Batch size	32
Max epochs	8 (early stopping patience = 3)
Warmup ratio	0.1
Weight decay	0.01
Best model metric	F1 Weighted
Split	80/20 stratified by topic_l1
Train samples	1,636
Test samples	410
Precision	fp16 (GPU)

4.3 Evaluation Results

L1 Classifier (8 classes)

95.85% Accuracy	95.97% F1 Weighted	94.07% F1 Macro	0.3587 Loss
---------------------------	------------------------------	---------------------------	-----------------------

Class	Precision	Recall	F1	Support
Entertainment	1.00	0.96	0.98	50
General	0.68	0.94	0.79	18
Geography	1.00	0.84	0.91	19
Health	0.98	1.00	0.99	42
History	1.00	0.90	0.95	21
Politics	0.97	0.93	0.95	92
Sports	0.98	0.98	0.98	90
Technology	0.96	0.99	0.97	78
Weighted avg	0.96	0.96	0.96	410

The General class has the lowest F1 (0.79) — expected, as small talk shares surface vocabulary with other domains. Precision of 0.68 means some General turns are being tagged domain-specific. All other classes are at $F1 \geq 0.91$.

L2 Classifier (19 classes)

94.88% Accuracy	94.95% F1 Weighted	94.76% F1 Macro	0.3731 Loss
---------------------------	------------------------------	---------------------------	-----------------------

Class	P	R	F1	Class	P	R	F1
AI	0.93	1.00	0.97	Mental Health	1.00	1.00	1.00
Bollywood	1.00	0.96	0.98	Nutrition	1.00	0.92	0.96
Cities	1.00	1.00	1.00	Olympics	1.00	0.80	0.89
Countries	0.92	0.86	0.89	Space	1.00	0.96	0.98
Cricket	0.82	1.00	0.90	Tennis	1.00	0.92	0.96
Fitness	0.95	1.00	0.97	UK	1.00	0.87	0.93
Football	1.00	1.00	1.00	USA	0.95	0.93	0.94
Gadgets	1.00	0.90	0.95	World Wars	1.00	0.88	0.93
General	0.74	0.94	0.83	Hollywood	1.00	0.96	0.98
India	0.93	0.98	0.95	Weighted avg	0.95	0.95	0.95

Cities, Football, and Mental Health achieved perfect F1. Weakest: General (0.83) and Cricket precision (0.82 — some Cricket turns pulled toward other Sports sub-topics).

5. Full Inference Pipeline

The full end-to-end inference pipeline was first developed and tested in `expansion_pipeline_FULL_INFERENCE.ipynb` using Llama-3.2-1B-Instruct loaded locally on GPU. The same pipeline logic was then ported to the Streamlit app with Groq API replacing the local LLM for deployment on CPU.

Pipeline Components	
spaCy NER	<code>en_core_web_trf</code> (notebook) / <code>en_core_web_sm</code> (Streamlit). Extracts PERSON, GPE, ORG, EVENT, NORP entities from every turn into a sliding-window Entity Register. Pruned to the last 20 messages.
Interruption detector	Regex patterns + word-count heuristics. Catches: <code>brb</code> , <code>brt</code> , <code>back</code> , <code>ok</code> , <code>wait</code> , <code>hold on</code> , <code>give me a sec</code> , etc. These bypass the LLM entirely and are tagged <code>General > General</code> .
LLM rewriting	Notebook: Llama-3.2-1B-Instruct via HuggingFace (local GPU). Streamlit: llama-3.1-8b-instant via Groq API. Same system prompt and output cleaning logic in both.
DistilBERT (L1+L2)	Both classifiers run on the expanded query. Output: topic label + confidence score per level. Run on CPU (<code>device=-1</code>) in both environments.

5.1 Expansion Prompt (used in both notebook and Streamlit)

```
Rewrite the latest user message into a fully self-contained query using context.
- Replace all pronouns and implicit references with explicit names.
- Expand incomplete questions, topic shifts, or comparisons into full sentences.
- Output ONLY the rewritten query. No quotes, no explanations.
Return as-is if already self-contained.
```

5.2 Expansion Examples

Raw user message	Context	Expanded query
"what are his duties?"	Prior: Narendra Modi is PM of India	"What are the duties of Narendra Modi as Prime Minister of India?"
"what about uk?"	Prior: discussing India PM	"Who is the Prime Minister of the United Kingdom?"
"compare both"	Prior: Modi and Rishi Sunak discussed	"Compare Narendra Modi and Rishi Sunak as Prime Ministers."
"brb"	(any context)	"brb" [tagged General, no expansion]
"back — tell me about cricket"	Prior: politics discussion	"Tell me about cricket." [topic switch detected]

6. Streamlit Demo App

6.1 Architecture

App Components	
LLM (expansion)	Groq API — llama-3.1-8b-instant. Replaces local Llama for CPU deployment. <1s latency.
NER	spaCy en_core_web_sm. Loaded via @st.cache_resource — initialises once per session.
Classifiers	Adignite/query-topic-l1-classifier + Adignite/query-topic-l2-classifier (DistilBERT). CPU, device=-1.
Context window	deque(maxlen=20) — exact implementation from the inference notebook.
Entity register	Rolling entity register across all turns. Pruned to last 20 messages.
Session state	All conversation state (history, entity register, turn index, results) stored in st.session_state.

6.2 Features

- Live conversation UI with chat bubbles (user left, assistant right)
- 3 pre-loaded sample conversations: Politics (India→UK), Sports (Cricket→Football), Technology (AI)
- 8 quick-input buttons for common test messages
- Optional bot response input to add assistant context to the Entity Register
- Per-turn result panel: raw → expanded query, L1/L2 topic pills with confidence bars, spaCy entity pills
- Turn history table (all turns, sortable)
- Session stats in sidebar: total turns, expansions performed, context window fill level
- Reset conversation button to clear all state

7. Repository Structure

7.1 Backend Pipeline Repo

 github.com/DeityAG/Real-Time-Query_Expansion_and_Topic-Tagging_backend_pipeline	
dataset_groq.ipynb	Stage 1 — synthetic data generation (Groq API)
merge_and_dedup.ipynb	Stage 2 — merge 3 CSVs, dedup (exact + BLEU)
bert_classifier.ipynb	Stage 3 — DistilBERT fine-tuning (L1 + L2)
expansion_pipeline_FULL_INFERENCE.ipynb	Stage 4 — full real-time inference
merged_conversations.csv	277 conversations (conversation-level)
turns_dataset.csv	2,046 turns (turn-level, BERT-ready)
DATASET_README.md	Dataset documentation
MODEL_README.md	Model documentation + eval results

7.2 Frontend Repo

 github.com/DeityAG/Real-Time-Query_Expansion_and_Topic-Tagging

├─

app.py

├─

requirements.txt

├─

.streamlit/

├─

├─ config.toml

├─

├─ secrets.toml

Main Streamlit application

Python dependencies

Dark theme configuration

API keys (not committed to git)

8. Key Design Decisions

Decision Rationale	
LLM for ALL dataset annotation	The LLM understands full conversation context during generation. Re-using it for annotation in a separate call produces higher-quality expansions than any post-hoc NER or rule-based approach on raw text.
NER only at inference (not generation)	At generation time, entities are implicitly guaranteed because conversations are seeded with a specific entity. The annotation LLM resolves all pronouns to that entity. NER is only needed at inference where no seed entity exists.
Expand BEFORE classify	Raw elliptical queries like “what about uk?” carry almost no signal for a classifier. The expanded form contains the domain, sub-topic, and named entity — giving DistilBERT the signal it needs.
Separate L1 and L2 classifiers	Keeping them independent lets each optimise for its label granularity. L2 (19 classes) has finer feature boundaries than L1 (8 classes), and a shared head would underperform on both.
Groq API for Streamlit deployment	Llama-3.2-1B on CPU takes 15–30s per turn. Groq’s llama-3.1-8b-instant (a larger model) returns in under 1 second via API. The only cost is a free Groq account.
DistilBERT over larger models	The classification task (~1.6k examples, 8 fixed classes) does not justify RoBERTa or BERT-large. DistilBERT achieves 95.85% L1 accuracy and runs on CPU in milliseconds — ideal for real-time inference.

9. Accuracy Summary

95.85%	94.88%	95.97%	94.95%
L1 Accuracy	L2 Accuracy	L1 F1 Weighted	L2 F1 Weighted

Both classifiers comfortably exceed the 95% accuracy target on the test set. The weakest performing class in both L1 and L2 is General (small talk), which is expected given its overlap with domain-specific vocabulary. All domain-specific classes (Politics, Sports, Technology, Health, etc.) perform at F1 ≥ 0.91 on L1 and ≥ 0.89 on L2.